

6교시: Hello World, nginx, 표준 실습 앱 첫 실행

수업 목표

- hello-world로 Docker client, daemon, registry, container 생성 흐름을 확인한다.
- nginx container를 port binding으로 실행하고 browser 또는 curl로 HTTP 응답을 확인한다.
- 실행, 확인, 로그, 중지, 삭제까지 80% hands-on 흐름으로 완료한다.

50분 흐름

시간	활동	비중	학생 산출
0-5분	실습 목표와 port 기준 확인	설명 10%	port note
5-15분	hello-world 실행	실행 20%	hello-world output
15-28분	nginx 실행과 HTTP 확인	실행 25%	curl -I evidence
28-38분	host HTML 수정 후 container 응답 변경 확인	실행 20%	bind mount evidence
38-44분	log 확인과 failure symptom 비교	실행 10%	log evidence
44-47분	중지/삭제/잔여 확인	실행 5%	cleanup evidence
47-50분	Day 2 Dockerfile 연결	설명 10%	readiness question

Visual 1: nginx 실행 검증



이 이미지는 `docker run -d -p 18080:80 nginx`로 container를 띄우고, `docker ps`, `browser`, `curl -I`, `docker logs`, `stop/rm`으로 검증하는 흐름을 보여준다. 본문 실습은 포트 충돌을 줄이기 위해 host port 18080을 사용한다.

Hands-on 1: hello-world 실행

```
docker run --rm hello-world
```

Linux 사전 테스트 결과

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
Digest:
sha256:0e760fdfbc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
 3. The Docker daemon created a new container from that image.
 4. The Docker daemon streamed that output to the Docker client.
```

이미 hello-world image가 있는 학생은 pull 관련 줄이 다르게 보일 수 있다. 핵심은 Hello from Docker!와 client/daemon/pull/create/output 흐름이다.

Hands-on 2: nginx container 실행

```
docker run -d --name paperclip-day1-nginx -p 18080:80 nginx:latest
docker ps --filter name=paperclip-day1-nginx
```

확인할 것

확인 항목	정상 기준
container name	paperclip-day1-nginx
image	nginx:latest
status	Up ...
port	0.0.0.0:18080->80/tcp 또는 [::]:18080->80/tcp

Hands-on 3: HTTP 응답 확인

browser에서 다음 주소를 연다.

```
http://localhost:18080
```

terminal에서는 다음 명령을 실행한다.

```
curl -I http://localhost:18080
```

Linux 사전 테스트 결과

```
HTTP/1.1 200 OK
Server: nginx/1.31.1
Date: Thu, 04 Jun 2026 04:07:34 GMT
Content-Type: text/html
Content-Length: 896
Connection: keep-alive
```

HTTP/1.1 200 OK가 핵심 evidence다. Date, Content-Length, nginx version은 실행 시점과 image version에 따라 다를 수 있다.

Hands-on 4: host 소스 수정 후 container 응답 변경 확인

Docker 내부 소스를 직접 고치는 장면을 보여주고 싶을 때 가장 먼저 구분해야 할 것이 있다. 실행 중인 container 안에 들어가 파일을 직접 수정하는 방식은 교육적으로는 흥미롭지만 운영 관점에서는 재현성이 약하다. container가 삭제되면 수정 내용이 사라질 수 있고, 누가 어떤 파일을 어떻게 바꿨는지 Dockerfile이나 README에 남지 않는다.

Day 1에서는 더 안전한 방식으로 "host 소스가 container 안에서 서빙되는 구조"를 본다. host의 HTML 파일을 nginx container의 /usr/share/nginx/html에 bind mount하고, host 파일을 수정한 뒤 browser/curl 응답이 바뀌는지 확인한다.

준비 파일

실습 파일은 [labs/nginx-html/index.html](https://github.com/leehyungja/labs/nginx-html/index.html)에 있다.

초기 내용의 핵심:

```
<h1>Docker nginx bind mount lab - v1</h1>
```

실행 명령

```
docker run -d \
  --name paperclip-day1-nginx-edit \
  -p 18081:80 \
  -v "$PWD/week2/day1/labs/nginx-html:/usr/share/nginx/html:ro" \
  nginx:latest

curl -s http://localhost:18081
```

Windows PowerShell 또는 macOS/Linux shell에 따라 현재 경로 표현은 달라질 수 있다. macOS/Linux terminal에서는 수업 repository root에서 위 명령을 실행한다. 경로가 맞지 않으면 pwd로 repository root를 확인한다.

Linux 사전 테스트 결과: v1

```
<h1>Docker nginx bind mount lab - v1</h1>
<p>이 파일은 host에서 수정하고 container 안의 nginx가 서빙한다.</p>
```

host 파일 수정

[labs/nginx-html/index.html](https://github.com/leehyungja/labs/nginx-html/index.html)의 핵심 줄을 다음처럼 바꾼다.

```
<h1>Docker nginx bind mount lab - v2</h1>
<p>host 파일을 수정했더니 container nginx 응답이 즉시 바뀐다.</p>
```

다시 확인한다.

```
curl -s http://localhost:18081
```

Linux 사전 테스트 결과: v2

```
<h1>Docker nginx bind mount lab - v2</h1>
<p>host 파일을 수정했더니 container nginx 응답이 즉시 바뀐다.</p>
```

이 결과는 image rebuild 없이 host file 변경이 container에서 서빙되는 것을 보여준다. Day 2의 Dockerfile에서는 bind mount가 아니라 image build로 파일을 image에 포함하는 방식을 배운다.

bind mount 정리

```
docker stop paperclip-day1-nginx-edit
docker rm paperclip-day1-nginx-edit
```

Hands-on 5: log 확인과 정리

```
docker logs paperclip-day1-nginx
docker stop paperclip-day1-nginx
docker rm paperclip-day1-nginx
docker ps --filter name=paperclip-day1-nginx
```

정리 후 docker ps가 헤더만 보여야 한다.

Failure drill: port 충돌

이미 18080을 쓰는 process나 container가 있으면 docker run이 실패할 수 있다. 이때 바로 port를 바꾸기 전에 먼저 원인을 확인한다.

증상	확인 명령	조치
port is already allocated	docker ps	기존 container stop/rm
browser 접속 실패	docker ps, curl -I	port binding 확인
container exited	docker ps -a, docker logs	log에서 원인 확인

Evidence 기록 양식

```
## Day 1 Lesson 6 Evidence
- hello-world 핵심 출력:
- nginx run command:
- docker ps PORTS:
- browser URL:
- curl status:
- docker logs 핵심 1줄:
- source edit result:
- cleanup result:
- blocker:
```

평가 기준

기준	2점 evidence
hello-world	client/daemon/pull/create/output 흐름을 설명했다.
nginx 실행	container name, image, port binding을 정확히 기록했다.
HTTP 검증	browser 또는 curl -I로 200 OK를 확인했다.
소스 변경 확인	bind mount된 HTML을 수정하고 container 응답이 바뀌는 것을 확인했다.
log 관찰	docker logs에서 nginx start evidence를 찾았다.
cleanup	stop/rm 후 running container가 없음을 확인했다.

공식 근거 링크

- Docker Docs: Run Docker Hub images, <https://docs.docker.com/get-started/docker-concepts/running-containers/>
- Docker Docs: Publishing and exposing ports, <https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/>
- Docker Docs: docker container logs, <https://docs.docker.com/reference/cli/docker/container/logs/>

다음 연결

다음 교시는 개인 면담과 환경 점검 시간이다. 방금 만든 evidence를 기준으로 막힌 학생은 증상, 명령, 출력, 가설, 필요한 도움을 정리한다.